More `constexpr` for `<cmath>` and `<complex>`

Document: P1383R0
Date: January 18, 2019
Project: Programming Language C++, Library Working Group
Audience: SG6 → LEWG → LWG
Reply to: Edward J. Rosten (`erosten@snap.com`) / Oliver J. Rosten (`oliver.rosten@gmail.com`)

Abstract

In [P0533], a scattering of `constexpr`, principally throughout `<cmath>`, has been proposed. This was subject to a constraint that the affected functions are limited to those which are, in a sense, no more complicated than the arithmetic operators $+, -, \times, /$. It is now time to lift this restriction thereby allowing a richer spectrum of mathematical functions to be used in a `constexpr` context.

CONTENTS

I. INTRODUCTION

Since its inception, `constexpr` has become an invaluable ingredient in compile-time programming. Indeed, part of its appeal is that the sharp distinction between meta-programming and runtime programming has in many instances become blurred. The interest in `constexpr` is reflected by the numerous papers proposing to increase the range of keywords and library functions that may be used in a `constexpr` context. As such, it is essential for the long-term uniformity of C++ that parts of the standard library are not left behind in this process.

This paper is the natural extension of [P0533] and seeks to significantly expand the number of functions in `<cmath>` (and also `<complex>`) which may be used in a `constexpr` context.

II. MOTIVATION & SCOPE

Prior to [P0533], no effort had been made to allow for functions in `<cmath>` to be declared `constexpr`; this despite there being glaring instances, such as `std::abs`, for which it is arguably perverse to still be unavailable for compile-time computation. Indeed, subsequent to [P0415R0], the situation has actually been better for `<complex>` than for `<cmath>`! The aim of [P0533] was to at least partially rectify this, while recognizing that attempting to completely resolve this issue in a single shot was too ambitious.

The broad strategy of [P0533] is to focus on those functions which are, in a sense, no more complicated than the arithmetic operators $+, -, \times, /$; the rationale for this being that the latter are already available in a `constexpr` context. As [P0533] has proceeded through the standardisation process, a collective desire has arisen to extend the scope to include a significant amount of what remains in `<cmath>`, in particular common mathematical functions such as `std::exp`. However, it seems too ambitious at this stage to include the mathematical special functions [sf.cmath] and so they are excluded from this proposal.

III. STATE OF THE ART AND IMPACT ON IMPLEMENTERS

With the exception of the special functions [sf.cmath], functions taking a pointer argument and those with an explicit dependence on the runtime rounding mode (see section IV B), gcc renders everything in `<cmath>` `constexpr`. While, on the one hand, [P0533] goes beyond gcc by proposing the functions such as

```
float modf(float value, float* iptr)
```

are constexpr, on the other it is more conservative since functions such as `exp` are excluded. This paper removes restrictions of the latter type.

Though clang does not have `constexpr` implementations, it does perform compile time evaluation of many mathematical functions (but not the special functions) during optimisation. The existence of compile time evaluation in GCC and clang demonstrates that implementation of this proposal is feasible.

IV. DESIGN DECISIONS

The parent paper [P0533] identifies two crucial issues that must be understood when rolling out `constexpr` in `<cmath>`: namely error flags and the rounding mode. It is, however, well worth emphasising that these issues must have been confronted when allowing $+, -, \times, /$ to be used in `constexpr` contexts. Therefore, it would be most disconcerting if this were to provide a barrier to rolling out `constexpr` in `<cmath>`; nevertheless, both error flags and rounding mode are worthy of a proper discussion, which interested readers may find in [P0533]. The next two subsections summarise the key points, after which we discuss the subtle issue of the interaction with the C runtime library. Finally, the key proposal of this paper is given.

A. Global Flags

There are various scenarios under which certain functions in `<cmath>` may set `errno` and/or raise exception flags, `FE_DIVBYZERO`, `FE_INVALID`, `FE_OVERFLOW`, `FE_UNDERFLOW` and `FE_INEXACT`. Drawing on the behaviour of $+, -, \times, /$ for inspiration, the proposal of [P0533] is as follows:

> Functions declared `constexpr`, *when used in a* `constexpr` *context*, should give a compiler error if division by zero, domain errors or overflows occur. When not used in a `constexpr` context, the various global flags should be set as normal.

At first sight, the distinction between these two contexts implies that many functions in `<cmath>` may not be declared `constexpr` as a pure library implementation. This is by no means the end of the world; however, the advent of `std::is_constant_evaluated()` [P0595] may mean that a pure library implementation is possible.

B. Rounding Mode

To facilitate discussion, [P0533] defines two types of rounding-mode dependence: weak and strong. To illustrate weak dependence, consider

```
constexpr double x{10.0/3.0}.
```
(4.1)

As an artefact of the limited precision of floating-point arithmetic, the answer depends on the rounding mode. However, this does not stop us from making definitions such as (4.1). Therefore, functions within `<cmath>` that exhibit such weak dependence are not ruled out from being declared `constexpr`.

This should be contrasted with strong dependence, which may be effectively illustrated by considering `float nearbyint(float x)`. The whole point of this function is to round depending on the runtime rounding mode, which may in principle be changed throughout a program's execution. It was therefore decided in [P0533] to exclude functions with strong dependence from being declared `constexpr` and this paper does not overturn that.

C. Interaction with the C standard Library

For a mathematical function which may be evaluated at translation time, it is desirable for there to be consistency with the values computed at runtime. However, the fact that the rounding mode may be changed at runtime indicates that this is not, in general, possible. Nevertheless, for weak rounding mode dependence the behaviour of $+, -, \times, /$ indicates that this is acceptable.

However, for more complicated mathematical functions there is an additional subtlety due to the interaction with the C standard library. In [library.c] it is noted that `<cmath>` makes available the facilities of the C standard library. One interpretation of this is that the C++ implementation could use one of several different C standard libraries. If so, constraining translation time behaviour so that it is consistent with the runtime behaviour could be difficult, quite apart from the issue of the runtime rounding mode.

This is sharpened by the following code snippet, kindly brought to our attention by Richard Smith:

```
#include <cmath>
double f() { return std::sin(1e100); }
```

For such code, any value in the range [-1, 1] could be considered reasonable and so it would not be expected to be consistent across library implementations. If it were possible to mark this function `constexpr`, then strong differences between translation time and runtime could emerge.

However, this is effectively already happening. It turns out that on clang (targeting x64), the following code is emitted:

```
.LCPI0_0:
        .quad        -4622843457162800295
_Z1fv:
        movsd        .LCPI0_0(%rip), %xmm0
        retq
```

with equivalent code generated by GCC.

This demonstrates that both compilers are already generating the results at translation time and, therefore, independently of the C library. For this particular example, it appears that current practise does indeed achieve consistency between translation time and runtime, though effectively by ignoring the latter!

The story does not end here. For more complicated examples and/or removing optimization, it may be that a call to the C library is made, after all. This implies that the value of, say, `std::sin(1e100)` evaluated in one part

of a code base may be very different from the (translation time) value evaluated elsewhere. Nevertheless, it seems reasonable in our opinion that both clang and GCC tacitly allow this. The example we are using, while instructive, is somewhat artificial: it exploits the fact that the floating-point approximation to periodic function ceases to make much sense for arguments beyond a certain magnitude.

A conservative option for this proposal would be to simply remove the trigonometric functions from the list to which we apply `constexpr`. We prefer not to do this but do acknowledge it as a possibility. On the one hand, allowing `constexpr` trigonometric functions is consistent with the existing behaviour of at least two major compilers (we have not checked beyond these). On the other, we think that preventing this on the basis of edge cases which amount to something of an abuse of floating-point would be the tail wagging the dog.

D. Conditions for `constexpr`

The conclusion of [P0533] was that two conditions may be used to systematically choose which functions in `<cmath>` should be declared `constexpr`. We reproduce these here, striking out the original form of the first, since this paper proposes removing it as a restriction:

Proposal. *A function in* `<cmath>` *shall be declared* `constexpr` *if and only if:*

1. ~~*When taken to act on the set of rational numbers, the function is closed (excluding division by zero);*~~

   The function does not belong to [sf.cmath];

2. *The function is not strongly dependent on the rounding mode.*

V. IMPACT ON THE STANDARD

This proposal amounts to a (further) liberal sprinkling of `constexpr` in `<cmath>`, together with a smattering in `<complex>`. It may be possible to do this as a pure library extension if use is made of [P0595] but even without this it is hoped for the burden on compiler vendors to be minimal.

VI. FUTURE DIRECTIONS

Ultimately it would be desirable to extend `constexpr` to some, if not all, of the special functions.

ACKNOWLEDGMENTS

We would like to thank Richard Smith for his usual perceptive comments.

REFERENCES

[P0533] Edward J. Rosten and Oliver J. Rosten, `constexpr` for `<cmath>` and `<cstdlib>`.

[P0415R0] Antony Polukhin, Constexpr for std::complex.

[P0595] Richard Smith, Andrew Sutton and Daveed Vandevoorde, `std::is_constant_evaluated()`.

[N4791] Richard Smith, ed., Working Draft, Standard for Programming Language C++.

VII. PROPOSED WORDING

The following proposed changes refer to the Working Paper [N4791]. Highlighting in red indicates changes proposed in this paper, whilst green indicates changes proposed in the companion paper, [P0533].

A. Modifications to "Header <complex> synposis" [complex.syn]

// [complex.value.ops], *values*

```
template<class T> constexpr T real(const complex<T>&);
template<class T> constexpr T imag(const complex<T>&);

template<class T> constexpr T abs(const complex<T>&);
template<class T> constexpr T arg(const complex<T>&);
template<class T> constexpr T norm(const complex<T>&);

template<class T> constexpr conj(const complex<T>&);
template<class T> constexpr proj(const complex<T>&);
template<class T> constexpr polar(const T&, const T& = T());
```

// [complex.transcendentals], *transcendentals*

```
template<class T> constexpr complex<T> acos(const complex<T>&);
template<class T> constexpr complex<T> asin(const complex<T>&);
template<class T> constexpr complex<T> atan(const complex<T>&);

template<class T> constexpr complex<T> acosh(const complex<T>&);
template<class T> constexpr complex<T> asinh(const complex<T>&);
template<class T> constexpr complex<T> atanh(const complex<T>&);

template<class T> constexpr complex<T> cos (const complex<T>&);
template<class T> constexpr complex<T> cosh (const complex<T>&);
template<class T> constexpr complex<T> exp (const complex<T>&);
template<class T> constexpr complex<T> log (const complex<T>&);
template<class T> constexpr complex<T> log10(const complex<T>&);

template<class T> constexpr complex<T> pow (const complex<T>&, const T&);
template<class T> constexpr complex<T> pow (const complex<T>&, const complex<T>&);
template<class T> constexpr complex<T> pow (const T&, const complex<T>&);

template<class T> constexpr complex<T> sin (const complex<T>&);
template<class T> constexpr complex<T> sinh (const complex<T>&);
template<class T> constexpr complex<T> sqrt (const complex<T>&);
template<class T> constexpr complex<T> tan (const complex<T>&);
template<class T> constexpr complex<T> tanh (const complex<T>&);
```

B. Modifications to "Header <cmath> synopsis" [cmath.syn]

```
namespace std{

...

constexpr float acos(float x); // see [library.c]
constexpr double acos(double x);
constexpr long double acos(long double x); // see [library.c]
constexpr float acosf(float x);
constexpr long double acosl(long double x);

constexpr float asin(float x); // see [library.c]
```

```
constexpr double asin(double x);
constexpr long double asin(long double x); // see [library.c]
constexpr float asinf(float x);
constexpr long double asinl(long double x);

constexpr float atan(float x); // see [library.c]
constexpr double atan(double x);
constexpr long double atan(long double x); // see [library.c]
constexpr float atanf(float x);
constexpr long double atanl(long double x);

constexpr float atan2(float y, float x); // see [library.c]
constexpr double atan2(double y, double x);
constexpr long double atan2(long double y, long double x); // see [library.c]
constexpr float atan2f(float y, float x);
constexpr long double atan2l(long double y, long double x);

constexpr float cos(float x); // see [library.c]
constexpr double cos(double x);
constexpr long double cos(long double x); // see [library.c]
constexpr float cosf(float x);
constexpr long double cosl(long double x);

constexpr float sin(float x); // see [library.c]
constexpr double sin(double x);
constexpr long double sin(long double x); // see [library.c]
constexpr float sinf(float x);
constexpr long double sinl(long double x);

constexpr float tan(float x); // see [library.c]
constexpr double tan(double x);
constexpr long double tan(long double x); // see [library.c]
constexpr float tanf(float x);
constexpr long double tanl(long double x);

constexpr float acosh(float x); // see [library.c]
constexpr double acosh(double x);
constexpr long double acosh(long double x); // see [library.c]
constexpr float acoshf(float x);
constexpr long double acoshl(long double x);

constexpr float asinh(float x); // see [library.c]
constexpr double asinh(double x);
constexpr long double asinh(long double x); // see [library.c]
constexpr float asinhf(float x);
constexpr long double asinhl(long double x);

constexpr float atanh(float x); // see [library.c]
constexpr double atanh(double x);
constexpr long double atanh(long double x); // see [library.c]
constexpr float atanhf(float x);
constexpr long double atanhl(long double x);

constexpr float cosh(float x); // see [library.c]
constexpr double cosh(double x);
constexpr long double cosh(long double x); // see [library.c]
constexpr float coshf(float x);
constexpr long double coshl(long double x);

constexpr float sinh(float x); // see [library.c]
constexpr double sinh(double x);
```

```
constexpr long double sinh(long double x); // see [library.c]
constexpr float sinhf(float x);
constexpr long double sinhl(long double x);

constexpr float tanh(float x); // see [library.c]
constexpr double tanh(double x);
constexpr long double tanh(long double x); // see [library.c]
constexpr float tanhf(float x);
constexpr long double tanhl(long double x);

constexpr float exp(float x); // see [library.c]
constexpr double exp(double x);
constexpr long double exp(long double x); // see [library.c]
constexpr float expf(float x);
constexpr long double expl(long double x);

constexpr float exp2(float x); // see [library.c]
constexpr double exp2(double x);
constexpr long double exp2(long double x); // see [library.c]
constexpr float exp2f(float x);
constexpr long double exp2l(long double x);

constexpr float expm1(float x); // see [library.c]
constexpr double expm1(double x);
constexpr long double expm1(long double x); // see [library.c]
constexpr float expm1f(float x);
constexpr long double expm1l(long double x);

constexpr float frexp(float value, int* exp); // see [library.c]
constexpr double frexp(double value, int* exp);
constexpr long double frexp(long double value, int* exp); // see [library.c]
constexpr float frexpf(float value, int* exp);
constexpr long double frexpl(long double value, int* exp);

constexpr int ilogb(float x); // see [library.c]
constexpr int ilogb(double x);
constexpr int ilogb(long double x); // see [library.c]
constexpr int ilogbf(float x);
constexpr int ilogbl(long double x);

constexpr float ldexp(float x, int exp); // see [library.c]
constexpr double ldexp(double x, int exp);
constexpr long double ldexp(long double x, int exp); // see [library.c]
constexpr float ldexpf(float x, int exp);
constexpr long double ldexpl(long double x, int exp);

constexpr float log(float x); // see [library.c]
constexpr double log(double x);
constexpr long double log(long double x); // see [library.c]
constexpr float logf(float x);
constexpr long double logl(long double x);

constexpr float log10(float x); // see [library.c]
constexpr double log10(double x);
constexpr long double log10(long double x); // see [library.c]
constexpr float log10f(float x);
constexpr long double log10l(long double x);

constexpr float log1p(float x); // see [library.c]
constexpr double log1p(double x);
constexprlong double log1p(long double x); // see [library.c]
```

```
constexpr float log1pf(float x);
constexpr long double log1pl(long double x);

constexpr float log2(float x); // see [library.c]
constexpr double log2(double x);
constexpr long double log2(long double x); // see [library.c]
constexpr float log2f(float x);
constexpr long double log2l(long double x);

constexpr float logb(float x); // see [library.c]
constexpr double logb(double x);
constexpr long double logb(long double x); // see [library.c]
constexpr float logbf(float x);
constexpr long double logbl(long double x);

constexpr float modf(float value, float* iptr); // see [library.c]
constexpr double modf(double value, double* iptr);
constexpr long double modf(long double value, long double* iptr); // see [library.c]
constexpr float modff(float value, float* iptr);
constexpr long double modfl(long double value, long double* iptr);

constexpr float scalbn(float x, int n); // see [library.c]
constexpr double scalbn(double x, int n);
constexpr long double scalbn(long double x, int n); // see [library.c]
constexpr float scalbnf(float x, int n);
constexpr long double scalbnl(long double x, int n);

constexpr float scalbln(float x, long int n); // see [library.c]
constexpr double scalbln(double x, long int n);
constexpr long double scalbln(long double x, long int n); // see [library.c]
constexpr float scalblnf(float x, long int n);
constexpr long double scalblnl(long double x, long int n);

constexpr float cbrt(float x); // see [library.c]
constexpr double cbrt(double x);
constexpr long double cbrt(long double x); // see [library.c]
constexpr float cbrtf(float x);
constexpr long double cbrtl(long double x);

// [c.math.abs], absolute values
constexpr int abs(int j);
constexpr long int abs(long int j);
constexpr long long int abs(long long int j);
constexpr float abs(float j);
constexpr double abs(double j);
constexprlong double abs(long double j);

constexpr float fabs(float x); // see [library.c]
constexpr double fabs(double x);
constexpr long double fabs(long double x); // see [library.c]
constexpr float fabsf(float x);
constexpr long double fabsl(long double x);

constexpr float hypot(float x, float y); // see [library.c]
constexpr double hypot(double x, double y);
constexpr long double hypot(long double x, long double y); // see [library.c]
constexpr float hypotf(float x, float y);
constexpr long double hypotl(long double x, long double y);

// [c.math.hypot3], three-dimensional hypotenuse
constexpr float hypot(float x, float y, float z);
```

```
constexpr double hypot(double x, double y, double z);
constexpr long double hypot(long double x, long double y, long double z);

constexpr float pow(float x, float y); // see [library.c]
constexpr double pow(double x, double y);
constexpr long double pow(double x, double y); // see [library.c]
constexpr float powf(float x, float y);
constexpr long double powl(long double x, long double y);

constexpr float sqrt(float x); // see [library.c]
constexpr double sqrt(double x);
constexpr long double sqrt(double x); // see [library.c]
constexpr float sqrtf(float x);
constexpr long double sqrtl(long double x);

constexpr float erf(float x); // see [library.c]
constexpr double erf(double x);
constexpr long double erf(long double x); // see [library.c]
constexpr float erff(float x);
constexpr long double erfl(long double x);

constexpr float erfc(float x); // see [library.c]
constexpr double erfc(double x);
constexpr long double erfc(long double x); // see [library.c]
constexpr float erfcf(float x);
constexpr long double erfcl(long double x);

constexpr float lgamma(float x); // see [library.c]
constexpr double lgamma(double x);
constexpr long double lgamma(long double x); // see [library.c]
constexpr float lgammaf(float x);
constexpr long double lgammal(long double x);

constexpr float tgamma(float x); // see [library.c]
constexpr double tgamma(double x);
constexpr long double tgamma(long double x); // see [library.c]
constexpr float tgammaf(float x);
constexpr long double tgammal(long double x);

constexpr float ceil(float x); // see [library.c]
constexpr double ceil(double x);
constexpr long double ceil(long double x); // see [library.c]
constexpr float ceilf(float x);
constexpr long double ceill(long double x);

constexpr float floor(float x); // see [library.c]
constexpr double floor(double x);
constexpr long double floor(long double x); // see [library.c]
constexpr float floorf(float x);
constexpr long double floorl(long double x);

float nearbyint(float x); // see [library.c]
double nearbyint(double x);
long double nearbyint(long double x); // see [library.c]
float nearbyintf(float x);
long double nearbyintl(long double x);

float rint(float x); // see [library.c]
double rint(double x);
long double rint(long double x); // see [library.c]
float rintf(float x);
```

```
long double rintl(long double x);

long int lrint(float x); // see [library.c]
long int lrint(double x);
long int lrint(long double x); // see [library.c]
long int lrintf(float x);
long int lrintl(long double x);

long long int llrint(float x); // see [library.c]
long long int llrint(double x);
long long int llrint(long double x); // see [library.c]
long long int llrintf(float x);
long long int llrintl(long double x);

constexpr float round(float x); // see [library.c]
constexpr double round(double x);
constexpr long double round(long double x); // see [library.c]
constexpr float roundf(float x);
constexpr long double roundl(long double x);

constexpr long int lround(float x); // see [library.c]
constexpr long int lround(double x);
constexpr long int lround(long double x); // see [library.c]
constexpr long int lroundf(float x);
constexpr long int lroundl(long double x);

constexpr long long int llround(float x); // see [library.c]
constexpr long long int llround(double x);
constexpr long long int llround(long double x); // see [library.c]
constexpr long long int llroundf(float x);
constexpr long long int llroundl(long double x);

constexpr float trunc(float x); // see [library.c]
constexpr double trunc(double x);
constexpr long double trunc(long double x); // see [library.c]
constexpr float truncf(float x);
constexpr long double truncl(long double x);

constexpr float fmod(float x, float y); // see [library.c]
constexpr double fmod(double x, double y);
constexpr long double fmod(long double x, long double y); // see [library.c]
constexpr float fmodf(float x, float y);
constexpr long double fmodl(long double x, long double y);

constexpr float remainder(float x, float y); // see [library.c]
constexpr double remainder(double x, double y);
constexpr long double remainder(long double x, long double y); // see [library.c]
constexpr float remainderf(float x, float y);
constexpr long double remainderl(long double x, long double y);

constexpr float remquo(float x, float y, int* quo); // see [library.c]
constexpr double remquo(double x, double y, int* quo);
constexpr long double remquo(long double x, long double y, int* quo); // see [library.c]
constexpr float remquof(float x, float y, int* quo);
constexpr long double remquol(long double x, long double y, int* quo);

constexpr float copysign(float x, float y); // see [library.c]
constexpr double copysign(double x, double y);
constexpr long double copysign(long double x, long double y); // see [library.c]
constexpr float copysignf(float x, float y);
constexpr long double copysignl(long double x, long double y);
```

```
double nan(const char* tagp);
float nanf(const char* tagp);
long double nanl(const char* tagp);

constexpr float nextafter(float x, float y); // see [library.c]
constexpr double nextafter(double x, double y);
constexpr long double nextafter(long double x, long double y); // see [library.c]
constexpr float nextafterf(float x, float y);
constexpr long double nextafterl(long double x, long double y);

constexpr float nexttoward(float x, long double y); // see [library.c]
constexpr double nexttoward(double x, long double y);
constexpr long double nexttoward(long double x, long double y); // see [library.c]
constexpr float nexttowardf(float x, long double y);
constexpr long double nexttowardl(long double x, long double y);

constexpr float fdim(float x, float y); // see [library.c]
constexpr double fdim(double x, double y);
constexpr long double fdim(long double x, long double y); // see [library.c]
constexpr float fdimf(float x, float y);
constexpr long double fdiml(long double x, long double y);

constexpr float fmax(float x, float y); // see [library.c]
constexpr double fmax(double x, double y);
constexpr long double fmax(long double x, long double y); // see [library.c]
constexpr float fmaxf(float x, float y);
constexpr long double fmaxl(long double x, long double y);

constexpr float fmin(float x, float y); // see [library.c]
constexpr double fmin(double x, double y);
constexpr long double fmin(long double x, long double y); // see [library.c]
constexpr float fminf(float x, float y);
constexpr long double fminl(long double x, long double y);

constexpr float fma(float x, float y, float z); // see [library.c]
constexpr double fma(double x, double y, double z);
constexpr long double fma(long double x, long double y, long double z); // see [library.c]
constexpr float fmaf(float x, float y, float z);
constexpr long double fmal(long double x, long double y, long double z);

// [c.math.fpclass], classification / comparison functions:
constexpr int fpclassify(float x);
constexpr int fpclassify(double x);
constexpr int fpclassify(long double x);

constexpr int isfinite(float x);
constexpr int isfinite(double x);
constexpr int isfinite(long double x);

constexpr int isinf(float x);
constexpr int isinf(double x);
constexpr int isinf(long double x);

constexpr int isnan(float x);
constexpr int isnan(double x);
constexpr int isnan(long double x);

constexpr int isnormal(float x);
constexpr int isnormal(double x);
constexpr int isnormal(long double x);
```

```
constexpr int signbit(float x);
constexpr int signbit(double x);
constexpr int signbit(long double x);

constexpr int isgreater(float x, float y);
constexpr int isgreater(double x, double y);
constexpr int isgreater(long double x, long double y);

constexpr int isgreaterequal(float x, float y);
constexpr int isgreaterequal(double x, double y);
constexpr int isgreaterequal(long double x, long double y);

constexpr int isless(float x, float y);
constexpr int isless(double x, double y);
constexpr int isless(long double x, long double y);

constexpr int islessequal(float x, float y);
constexpr int islessequal(double x, double y);
constexpr int islessequal(long double x, long double y);

constexpr int islessgreater(float x, float y);
constexpr int islessgreater(double x, double y);
constexpr int islessgreater(long double x, long double y);

constexpr int isunordered(float x, float y);
constexpr int isunordered(double x, double y);
constexpr int isunordered(long double x, long double y);
```

C. Modifications to "Three-dimensional hypotenuse" [c.math.hpot3]

```
constexpr float hypot(float x, float y);
constexpr double hypot(double x, double y);
constexpr long double hypot(double x, double y);
```